



Curso de Python

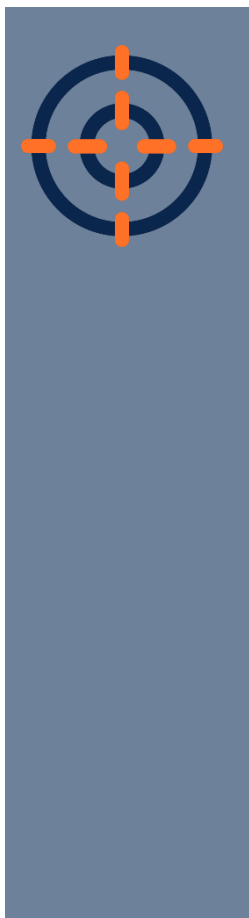
Unidad 2. Fundamentos de programación en Python.

ÍNDICE

Contenido

1. Introducción.....	4
2. Variables, tipos de datos y estructuras de control de flujo.....	5
2.1 Tipos de datos	9
2.1.1 Tipos de datos enteros (int)	10
2.1.2 Tipo de datos flotantes (float)	12
2.1.3 Tipo de datos lógicos (booleanos).....	13
2.1.4 Números complejos.....	13
2.1.5 Conversión entre tipos	14
2.1.6 Cadenas de caracteres.....	15
2.2 Estructuras de control de flujo.....	16
2.2.1 Condicionales	16
Testigo	26
Contador	28
Acumulador	28
Bucles anidados (variables independientes)	29
Bucles anidados (variables dependientes)	30
Bucle WHILE.....	31
2.3 Funciones	32
2.3.1 Funciones sin parámetros	32
2.3.2 Funciones con parámetros	39
2.4 Ejercicios	45
2.5 Métodos.....	46
2.6 Listas, tuplas y diccionarios.	46
2.6.1 Listas	46
2.6.2 Tuplas	49

2.6.3 Diccionarios	50
3 Resumen.....	54



Objetivos

En esta unidad aprenderás a:

- Conocer las partes principales de un programa en python
- Variables, tipos de datos. Dónde guardamos la información
- Estructuras de control de flujo. Cuando repetir sí es necesario.
- Funciones y métodos. Lo que se repite muchas veces.
- Listas, tuplas y diccionarios. Otras formas de guardar la información

1. Introducción

Hasta el momento hemos conocido las herramientas que nos ayudan a crear un programa en Python. Ahora “sólo” nos falta aprender a realizar estas aplicaciones a partir de dichas herramientas.

Lo que aprendas en este tema representa el 90% de lo que incluye un programa en Python. Por lo tanto, es muy importante que queden claros los conceptos que se incluyen en esta Unidad.

Para ello, te plantearé una serie de programas para que puedas practicar los conceptos aprendidos.

Conocerás:

- qué es una variable y para qué sirve (siempre se usan).
- Para que queremos las estructuras de control de flujo, ya que un programa, cuando se ejecuta, no se recorre de arriba debajo de manera consecutiva.
- Cuando algo se repite muchas veces (como por ejemplo cuando queremos saber qué hora es) crearemos funciones, las cuales nos facilitarán las cosas.
- Cuando queremos guardar mucha información, y trabajar con ella, utilizaremos otra serie de estructuras, como las listas. Que harán que obtener información de estos datos sea más sencillo.

Así que, vamos con todo.

2. Variables, tipos de datos y estructuras de control de flujo

Ponte en el supuesto de que estás rellenando los datos de un formulario en una página web y te piden una serie de información. Lo típico que te piden como tu dirección o fecha de nacimiento. Toda esa información que has de rellenar ha de guardarse, y es ahí donde entran en juego las variables.

Podemos también pensar en una variable como en un cajón donde se puede guardar algo. Ese algo es información.

A la hora de crear esos “cajones”, hay que decir que Python es un lenguaje de “tipado dinámico”. Qué quiere decir esto. Que si yo por ejemplo quiero crear una variable para guardar la fecha de nacimiento del usuario, no es necesario que la cree previamente. En otros lenguajes sí que es necesario hacer esto. Es decir, que habría que indicarle a nuestro programa que en la variable `fecha_de_nacimiento` (siendo originales con el nombre), vamos a guardar un dato que va a ser una fecha.

Python se adaptará a los valores que vayamos guardando en las variables (ya habrás adivinado por qué se llaman variables, porque su valor va variando a lo largo de la ejecución del programa). Así, Python asume que serán de un tipo u otro según el tipo de valor dado.

En lo siguientes puntos conocerás los distintos tipos de variables que existen. Ya que su utilización dependerá de lo que queramos guardar en ellas.

Es lógico que cuando guardemos un precio, este valor tenga decimales (los céntimos) o si vamos a guardar el número de huevos que hemos comprado, ese valor no tenga decimales.

Antes de conocer los diferentes tipos, probemos un ejemplo, siempre dentro de Thonny.

Escribimos los siguiente:

```
<sin nombre> * ×  
1 total=5  
2 print(total)
```

En la instrucción en la línea 1, tenemos la forma en que se guardan los valores en una variable. No confundir con una comparación (eso ya lo aprenderemos). Y en la línea 2 mostramos el contenido de la variable total. Y si lo ejecutamos (pulsando play verde o F5) nos mostrará un 5 por la consola.

¿Qué valor tendrá x después de las siguientes órdenes?

```
x = 5  
x = 3
```

La respuesta correcta es que las variables se SOBREESCRIBEN, así que toman el último valor que se les asigne. En nuestro caso, x valdría 3.

Dime ahora, ¿qué pasa después de estas líneas de código?

```
x = 3  
  
x = x + 1
```

Cuando empecé con la programación la última línea me hacía estallar la cabeza:

¿Cómo va a ser posible que x “sea igual a” x+1? ¡¡¡¡Es IMPOSIBLE!!!!

Pero ya os he dicho que EL IGUAL EN PROGRAMACIÓN ES UNA ASIGNACIÓN.

x = x + 1 se lee así:

“La variable x tomará el valor que tiene ahora más una unidad”

Así que después de las dos líneas de código que escribimos, x valdrá 4.

Estas expresiones son muy comunes y debes familiarizarte con ellas.

Recordemos una frase que decíamos antes: Cada vez que tengamos que recordar un valor necesitamos una variable.

Te propongo hacer un programa que intercambie el valor de dos variables. Éste ya es un programa con cierto nivel.

El primer intento que haría un principiante (ojo, está mal) es posible que fuera este:

```
a = 3
b = 4
print('El valor de a es ', a)
print('El valor de b es ', b)
a=b
b=a
print('El valor de a es ', a)
print('El valor de b es ', b)
Pero esto es lo que obtengo en la consola
```

(Antes del cambio)

```
El valor de a es 3
El valor de b es 4
```

(Después del cambio)

```
El valor de a es 4
El valor de b es 4
```

De nuevo, un ejemplo de programa con la sintaxis correcta, pero que no actúa

como debe. Recuerda, un programa sin errores sintácticos no es un programa que haya probado que funciona adecuadamente.

Veamos qué ha pasado, instrucción por instrucción y viendo qué valores están tomando las variables después de cada paso.

Instrucción	Valor de a	Valor de b	Comentarios
a = 3	3	No existe aún	Asignación de a
b = 4	3	4	Asignación de b
a = b	4	4	a toma el valor que tenía b b no cambia su valor
b = a	4	4	b toma el valor que tenía a en el paso anterior, o sea, 4 a no cambia su valor

Como puedes ver, el valor primitivo de a se pierde, porque no podemos hacer el intercambio “a la vez”, sólo instrucción por instrucción. Por eso necesitamos guardar el valor primitivo de a, antes de sobrescribirlo, para poder luego pasárselo a b.

Y, cuando necesitamos guardar algo... necesitamos una variable.

La llamaremos temp, nombre típico de variables temporales que no queremos usar más que para cosas intermedias. También es frecuente usar aux (auxiliar).

Veamos si funciona.

```
a = 3
b = 4
print('Antes del cambio')
print('El valor de a es ', a)
print('El valor de b es ', b)
temp = a #guardamos el valor de a en la variable temp
a = b #asignamos a la variable a al valor que tiene b
b = temp #asignamos a b el valor que tenía a que estaba guardado en
la variable temp
print('Después del cambio')
print('El valor de a es ', a)
print('El valor de b es ', b)
```

Si lo ejecutáis, en la consola aparece

Antes del cambio

El valor de a es 3

El valor de b es 4

Después del cambio

El valor de a es 4

El valor de b es 3

¡Ahora sí funciona!

Si analizamos de nuevo el valor de las variables en cada paso lo verás más claro

Instrucción	Valor de a	Valor de b	Valor de temp	Comentarios
a=3	3	No existe aún	No existe aún	
b=4	3	4	No existe aún	
temp=a	3	4	3	Guardamos valor de a en temp
a=b	4	4	3	Sobreescribimos a
b=temp	4	3	3	Recuperamos el valor que tenía a para asignárselo a b, gracias a que estaba guardado en temp

2.1 Tipos de datos

Antes de conocer los distintos tipos de datos que existen para guardar información, es necesario que conozcas las reglas que deben cumplir las variables para poder utilizarse:

- No pueden utilizarse palabras reservadas del propio Python como variables. Es decir, no se puede utilizar la palabra print como variable (existen muchas más)
- No deben incluirse espacios. edad usuario no sería un nombre válido.
- Sólo pueden contener letras, números y el guión bajo _
- Tampoco pueden comenzar con un número.

and	as	assert	break	class	continue
def	del	elif	else	except	exec
finally	for	from	global	if	import
in	is	lambda	nonlocal	not	or
pass	raise	return	try	while	with
yield	True	False	None		

Éstas son las palabras reservadas en Python que no pueden ser utilizadas como variables.

De todas formas, si creamos una variable como no toca, ya se encargará el intérprete de avisarnos.

SIEMPRE, utiliza nombres de variables relacionadas con lo que van a contener: resultado, fecha_de_nacimiento, dirección, etc

2.1.1 Tipos de datos enteros (int)

Se corresponden con los números enteros en matemáticas. Utilizaremos este tipo de datos cuando no haga falta el uso de decimales.

Cuando no sepamos el tipo que tiene una variable, podemos utilizar lo siguiente para saberlo (cuando nuestros programas se vuelvan complicados):

```
:sin nombre> * ×  
1 total=5  
2 print(type(total))
```

El resultado de ejecutar esto es:

```
Consola ×  
>>> %Run -c $EDITOR_CONTENT  
<class 'int'>  
>>>
```

Nos está indicando que la variable total es de clase int (en inglés entero se dice integer)

Las operaciones que podemos hacer con los números enteros son:

 sumar	
 restar	 división entera
 multiplicar	 módulo
 dividir	 potenciación

Los 4 primeros de la izquierda son conocidos. Los de la derecha son menos conocidos.

Probémoslos para conocerlos:

<sin nombre> * ×

```

1 numero1=5
2 numero2=2
3 print(numero1//numero2)
4 print(numero1%numero2)
5 print(numero1**numero2)

```

Consola ×

```

>>> %Run -c $EDITOR_CONTENT
2
1
25

```

División entera nos devuelve la parte entera del cociente, módulo es el resto de la división y potencia no necesita explicación.

A continuación, tienes un ejemplo que puedes probar, donde sumamos dos números y mostramos el resultado por pantalla. En este caso utilizaremos tres variables. No hay límites en el uso de variables, aunque hay que utilizar el número necesario para no incurrir en errores. Recuerda, que, en un futuro, nuestro programa puede contener decenas de variables, y a veces las reciclamos. Por lo que habrá que tener cuidado con ellas.

ejemplos.py ×

```

1 numero1=5
2 numero2=2
3 resultado=numero1+numero2
4 print('El resultado es ',resultado)

```

Debemos tener en cuenta que todas las entradas por teclado se toman como cadenas de caracteres.

En el ejemplo anterior, si usamos input para que el usuario nos escriba los dos números, deberemos convertirlos a números enteros antes de realizar las operaciones (recuerda que usábamos input para recibir datos por teclado).

```
ejemplos.py ×
1 numero1=input('Escribe el primer número ')
2 numero2=input('Escribe el segundo número ')
3 numero1=int(numero1)
4 numero2=int(numero2)
5 suma=numero1+numero2
6 print('La suma es ',suma)
7
```

Aquí hacemos la conversión de lo que el usuario introduce. Al ser numero1 una variable, podemos guardar ahí la conversión. Si el usuario tiene mala idea, en vez de un número podría insertar cualquier otro carácter, lo que daría un error en tiempo de ejecución. Ese tipo de comprobación se debería realizar.

Te dejo que pruebes esto.

2.1.2 Tipo de datos flotantes (float)

Este tipo de números son los que tienen una parte entera y una parte decimal. Se pueden aplicar las mismas operaciones que con los datos enteros.

```
ejemplos.py ×
1 precio=7.2
2 print(type(precio))
3
```

Prueba este ejemplo para comprobar cómo se utilizan los datos float. Date cuenta de que para poner los números decimales se utiliza un punto

¿Y qué pasa si sumamos un número entero y uno float? Que el intérprete convierte el resultado a float

```
ejemplos.py ×  
1 precio1=7.2  
2 precio2=5  
3 total=precio1+precio2  
4 print('El total es ',total)
```

Pruébalo

2.1.3 Tipo de datos lógicos (booleanos)

No todo van a ser números o letras. Imagina que queremos comparar las edades de dos personas. La respuesta de esa comparación va a ser Sí o No. ¿Laura es mayor que Marta? La respuesta no es un número, sino Sí o No. Es lo que se llama tipo booleano. También se representa el Sí con 1 y No con 0.

Más adelante conoceremos los condicionales en Python, pero adelantemos un ejemplo:

```
ejemplos.py ×  
1 edad_Laura=20  
2 edad_Marta=15  
3 if (edad_Laura>edad_Marta):  
4     print('Laura es mayor')
```

Aquí tienes un ejemplo donde se utilizan los booleanos.

```
Consola ×  
>>> %Run ejemplos.py  
Laura es mayor
```

Éste es el resultado de la ejecución (aunque lo normal sería preguntar las edades al usuario)

2.1.4 Números complejos

No son muy utilizados, pero debes saber que los tienes disponibles.

Al igual que en matemáticas, constan de una parte real y otra imaginaria. Esta parte imaginaria suele acompañarse de la letra j, aunque también puede usarse la i

`parte_real + parte_imaginaria*j`

Ejemplos:

`a = 5 + 5j`

`b = 1.3 - 7j`

Algún ejemplo (a la derecha tienes el resultado de la ejecución):

<code>ejemplos.py</code> × <pre> 1 c = 3 + 5j 2 print(c) 3 print(type(c)) </pre>	Consola × <pre> >>> %Run ejemplos.py (3+5j) <class 'complex'> </pre>
--	--

Aparte de los números imaginarios, existen en Python librerías (conjunto de programas y datos) que nos permiten trabajar con fracciones, raíces, etc Algunos de ellos los veremos más adelante.

2.1.5 Conversión entre tipos

Es posible convertir entre enteros, flotantes y booleanos. Así partiendo de un tipo de dato puedes llegar a otro tipo de dato. Algunas de estas conversiones son **implícitas**, es decir, que al operar entre dos tipos de datos distintos se consigue un dato de uno u otro tipo. Mientras que otros tipos de conversiones son **explícitas** las tienes que *declarar*. Para esto, tienes algunas funciones predefinidas, que te ayudarán a realizar estas conversiones:

`bool`

`int`

`float`

Estas tres funciones nos permiten convertir el contenido que les pasemos a booleano, entero y flotante respectivamente.

2.1.6 Cadenas de caracteres

Son un tipo de datos que permiten almacenar secuencias de caracteres. Es de los más utili Para crear una, es necesario incluir el texto entre comillas dobles "

ejemplos.py ×

```
1 saludo="Hola"  
2 print(saludo)
```

Obtendríamos el saludo por pantalla.

saludo es una cadena de caracteres

Existen numerosas funciones que nos permiten trabajar con las cadenas de caracteres (en adelante les llamaremos strings).

Podemos coger un trozo de la cadena, sustituir caracteres

ejemplos.py ×

```
1 comienzo="En un lugar de la Mancha"  
2 trozo=comienzo[0:5]  
3 print(trozo)  
4
```

En este ejemplo, coge el string comienzo y desde la posición 0 (en este caso es la letra E, no empieza por la posición 1) y coge 5 caracteres.

Imprimiría En un

Con el símbolo + concatenamos strings

Incluso, poniendo comienzo.upper() convertiríamos la cadena a mayúsculas.

Y existe la función len para obtener la longitud de una cadena de caracteres.

Longitud=len(comienzo)

En este caso serían 24 caracteres.

Ahora quizá es un buen momento para introducir los COMENTARIOS.

En este caso, el intérprete ignorará todo lo que va después de la almohadilla.

Utilizaremos los comentarios para explicar nuestro código, para saber lo que realiza en esa parte. Porque seguro que, si tenemos que revisar el programa más adelante, nosotros u otra persona, no nos acordaremos de lo que se hacía.

Por ejemplo:

#Aquí sumamos todas las cantidades y la guardamos en la base de datos

Al llegar a esa línea, el intérprete la pasa por alto. Da un poco de pereza hacerlo, pero son extremadamente útiles.

Prueba el siguiente ejemplo, que ya tiene un poco de todo de lo que hemos ido aprendiendo: conversiones, comentarios, lectura por teclado, operaciones o muestra de datos por pantalla

```
ejemplos.py x
1 print('¿Cómo te llamas?')
2 nombre = input()
3 #nombre usuario
4 print('¿En qué año naciste?')
5 year = int(input())
6 #int convierte en entero la entrada de teclado que de otro modo sería un string
7 edad = 2023 - year
8 #edad usuario (en el 2022)
9 print('Hola', nombre, 'tienes', edad, 'años')
```

2.2 Estructuras de control de flujo

Este tipo de estructuras hacen que cambie el flujo normal de un programa, que como ya hemos comentado siempre va de arriba a abajo.

Por ejemplo, no es igual si el usuario tiene ya 18 años (condición) o hay que mostrar algo varias veces por pantalla. Es muy útil que puedan tomar decisiones diferentes según sean los datos que reciben o los resultados que van obteniendo.

2.2.1 Condicionales

Por ejemplo, recordemos este programa nuestro que nos preguntaba el año de nacimiento y nos decía Hola Pepe tienes 47 años, podría ser divertido que nos diese diferentes respuestas según los años que nos calcule (por ejemplo si somos mayores de edad o no). Para eso necesitamos los CONDICIONALES.

Un condicional tiene tres “partes”.

La condición que queremos comprobar (en este caso $\text{edad} > 18$)

Las acciones que tomamos si se cumple la condición (nos dirá que somos viejos)

Las acciones que tomamos si NO se cumple la condición (dirá que somos jóvenes)

Probemos con el antiguo programa:

```
print('¿Cómo te llamas?')
nombre = input() #nombre usuario
print('¿En qué año naciste?')
year = int(input()) #int convierte en entero la entrada de
teclado que de otro modo sería un string
edad = 2023 - year #edad usuario (en el 2023)
print('Hola', nombre, 'tienes', edad, 'años')
```

Nos sirve todo, menos la línea final, ahí es donde vamos a meter el condicional y daremos otras salidas por pantalla diferentes a la que tenemos aquí.

No la borréis, os voy a enseñar un truco muy común cuando estamos probando un programa y queremos “activar” y “desactivar” líneas de código, para ver qué pasa cuando las quitamos.

¿Recordáis que, cuando poníamos la almohadilla(#), Python ignoraba lo que se escribía después y que nos servía para poner comentarios? Pues podemos poner una almohadilla delante de la línea que queremos anular temporalmente y Python la ignorará. A esto se le llama COMENTAR CÓDIGO.

Bien, vayamos al condicional

Debemos empezar con la palabra reservada `if` (por lo que una variable no se podrá llamar `if`)

Después ponemos la condición

Después añadimos dos puntos (`:`)

Haced esto y dadle a intro.

¿Veis que al cambiar de línea el cursor ha avanzado un poco?

Se llama **INDENTACIÓN** y sirve para tener claro que las líneas siguientes tienen que ver con el `if`, que es un pequeño bloque de código dentro de ese condicional.

En esa línea siguiente ponemos las instrucciones que queremos se ejecuten cuando se cumpla la condición (puede ser una o muchas)

```
if edad > 18:  
    print('Hola', nombre, 'eres muy viejo')
```

Prueba el código, recuerda que la ausencia de errores no asegura que funcione bien. Mete dos años de nacimiento diferentes, uno que nos dé mayor de edad y otro que no. Cubre todos los casos cuando hagas pruebas.

Como ves, en el caso de que se cumpla la condición, ejecuta el código “interior” del condicional y nos imprime este “simpático” mensaje.

Pero, en el caso de que **NO** se cumpla. No hace nada. Esto no es lo que queremos.

Para remediarlo, tenemos `else` (en caso contrario...).

Añadiríamos al programa que tenemos las siguientes líneas.

```
if edad>18:  
    print('Hola', nombre, 'eres muy viejo')  
else:  
    print('Hola', nombre, 'eres muy joven')
```

Date cuenta de que

- else está a la altura del if correspondiente
- después de else tenemos dos puntos porque también viene un código "interior"
- el código "interior" de else también está indentado.

Lo que escribimos justo después del if debe ser una EXPRESIÓN LÓGICA, algo que sea como una pregunta que pueda responderse con SÍ o NO, VERDADERO o FALSO. En nuestro ejemplo ¿Es edad mayor que 18? Efectivamente, puede responderse VERDADERO o FALSO.

También valdrían expresiones complejas. Por ejemplo, si queremos dejar pasar a un chaval a una atracción un poco peligrosa, quizá queramos que sea mayor que 10 años o que su altura sea mayor que un metro. Podríamos expresarlo así:

`edad > 10 or altura > 1.0`

Por ejemplo:

```
altura = 1.2
edad = 10
if edad > 9 or altura > 1.0:
    print('pasa')
else:
    print('no pasa')
```

Con la altura y la edad que he puesto, el programa me dice 'pasa', porque tiene más altura y más edad de la exigida.

Así que, funciona bien con un ejemplo y no da errores sintácticos, ¿puedo quedarme tranquilo? ¿Cuántos casos diferentes hay?

A mí me salen cuatro (refiriéndome a edad y altura):

menos-menos, menos-más, más-menos y más-más

Cambiad a mano los valores para ejemplificar los cuatro casos y ver que se comporta adecuadamente. Y esto, queridos míos, sí que es comprobar que un programa funciona debidamente.

De la misma forma que hemos usado "or" podemos usar "and" o "not" y crear

expresiones todo lo complejas que necesitemos.

Pero quería pararme un momento para resaltar un caso que suele dar problemas:

¿cómo escribir que quiero comprobar si la edad es igual a 18?

Lo primero que se nos ocurre es poner `if edad = 18`: pero esto está MAL.

Recordad que el IGUAL lo usábamos para ASIGNAR.

Para COMPARAR usaremos un DOBLE IGUAL (También son símbolos válidos para expresiones: mayor o igual (`>=`), menor o igual (`<=`) y distinto (`!=`))

Lo correcto es, por ejemplo:

```
print('Escribe tu nombre')
nombre = input()
if nombre == 'Pepe':
    print('Te llamas igual que yo!')
```

Ya conoces todo lo que hay en este programa, pero permíteme señalar unos detalles.

- Las cadenas de caracteres se escriben entre comillas
- PRINT no mira si lo que hay entre las comillas es código Python o español, o tiene faltas de ortografía como es el caso (falta el signo ¡), simplemente pone los caracteres que haya.

Volvámonos locos... pongamos un condicional dentro de otro, así tendremos lo que

llamamos, CONDICIONALES ANIDADOS.

Por ejemplo: Quiero celebrar mi cumpleaños y, por supuesto, me gustaría que vinieran más de cinco personas, pero no tengo sitio para más de diez.

```
'''El número de invitados no puede ser superior a 10 ni
inferior a 5'''
```

```
print('¿Cuántos invitados vienen')
invitados=int(input())
if invitados > 5:
if invitados < 10:
print('Hay fiesta') #caso entre 5 y 10
else:
print('No hay fiesta') #caso mayor que 10
else:
print('No hay fiesta') #caso menor que 5
```

Comprueba el funcionamiento con un valor de cada zona interesante (menos que cinco, entre cinco y diez, más de diez).

Analicemos el código.

Primer if, ¿estamos por encima de 5?, si es así ejecutamos el código interior, si no fuera así, nos iríamos al else que está a su altura (el último).

Si entramos en el primer if es que estamos por encima de 5, pero ahora nos preguntamos: sabiendo que estamos por encima de 5, ¿estamos por debajo de 10?

Si fuera verdad, estamos en la franja adecuada entre cinco y diez, y si no se cumple que seamos menor que diez, pasaríamos al else que está a la altura del segundo if.

Igual lo ves más claro, si entendemos por qué no funcionaría bien el código si no estuvieran anidados. Pongamos los dos if a la misma altura.

```
'''El número de invitados no puede ser superior a 10 ni
inferior a 5'''
print('¿Cuántos invitados vienen')
invitados = int(input())
if invitados > 5:
print('Casol')
```

```
else:  
print('Caso2')  
if invitados < 10:  
print('Caso3')  
else:  
print('Caso4')
```

¿Dónde pongo, “hay fiesta” y “no hay fiesta”? Da igual, no va a funcionar en ningún caso. Te pongo un ejemplo para que veas que “casca”. Prueba tú con otros, si crees que van a funcionar, pero recuerda probar los tres casos posibles.

Si los pongo así, por ejemplo:

```
'''El número de invitados no puede ser superior a 10 ni  
inferior a 5'''  
print('¿Cuántos invitados vienen')  
invitados = input()  
if invitados > 5:  
print('Hay fiesta')  
else:  
print('No hay fiesta')  
if invitados < 10:  
print('Hay fiesta')  
else:  
print('No hay fiesta')
```

Imagina que tengo 3 invitados.

El primer condicional me dirá que no hay fiesta, pero el segundo condicional me dice que sí, porque tres invitados es menor que diez. Al NO estar anidados no estoy mirando si es menor que diez SABIENDO ya que es mayor que cinco.

A veces abreviamos estos condicionales con el comando elif que sería else...if

Algo así como ¿Se cumple esta condición? No

Sabiendo que no se cumple la primera condición, ¿se cumple la segunda?

Por ejemplo, adivina un número entre uno, dos y tres.

```
'''Adivina un número entre 1, 2 y 3.  
El número a adivinar es el 2'''  
print('Dime un número entre el uno y el tres')  
test = input()  
if test == '1' or test == '3':  
    print('Has fallado')  
elif test == '2':  
    print('Has acertado')  
else:  
    print('No has escrito un número entre el uno y el tres')
```

Se leería así:

¿Es test igual a uno o a tres? Si es así, di que “ha fallado”.

(elif) SABIENDO que no es igual a uno o a tres, ¿es igual a dos? Si es así, di que “ha acertado”.

(else) En CUALQUIER OTRO CASO, di que no ha escrito bien lo que le pedíamos.

En este caso es sencillo porque las opciones son excluyentes, pero en otros casos hay que ser cuidadoso para escoger adecuadamente las condiciones y en el orden que se piden, quién se anida en quién...y, finalmente, hacer pruebas para todos los casos posibles.

Esto podría usarse para crear un “menú”.

Hagamos un programa que te pida dos números y te dé la posibilidad de elegir sumarlos, restarlos o multiplicarlos.

A continuación te muestro el resultado , pero puedes hacerlo por tu cuenta y luego comprobar que funciona. En el mundo de la programación, no suele haber siempre una solución única.

SOLUCIÓN:

```
'''Pide dos números, y te da a elegir sumarlos, restarlos o
multiplicarlos'''
print('Dime el primer número')
a = int(input())
print('Dime el segundo número')
b = int(input())
print('Elige suma(s), resta(r) o multiplicación(m)')
operacion = input()
if operacion == 's':
    print('El resultado es', a + b)
elif operacion == 'r':
    print('El resultado es', a - b)
elif operacion == 'm':
    print('El resultado es', a * b)
else:
    print('No has escogido correctamente la opción')
```

Ya tenemos una mini-calculadora programada.

Aunque ya todo es conocido, permíteme que te recuerde que:

- Para poder usar como números enteros los números que teclea el usuario hay que pasarlos por la función int porque si no los guardaría como string.
- Cuando miro qué ha escogido el usuario, lo comparo con 's', 'r' o 'm' porque son strings
- Es bueno poner una opción else final porque así, siempre ocurrirá algo. Nos aseguramos que el programa dará alguna respuesta.

2.2.2 Bucles

Llamamos bucles a la repetición de la ejecución de un bloque de instrucciones en la que cada repetición se llama iteración. En cada iteración debe cambiar algo del

programa (como el valor de alguna variable), para que no se entre en un bucle infinito y nos quedemos sin memoria (la del ordenador, claro).

Existen dos tipos de bucles, FOR y WHILE, y en cada uno de ellos debemos especificar lo siguiente:

- Punto de inicio del bucle.
- Punto de fin del bucle.
- Número de iteraciones.

Bucle FOR

Utilizaremos este tipo de bucles cuando sepamos el número exacto de iteraciones que se van a realizar en la ejecución.

Veamos un primer ejemplo sencillo para entender el funcionamiento:

```
ejemplos_bucles.py ×  
1 for i in range (0,100):  
2     print('Soy la vuelta ',i)  
3
```

Este primer ejemplo, lo que hace es imprimir esa frase 100 veces, empezando por la cero

```
Consola ×  
>>> %Run ejemplos_bucles.py  
Soy la vuelta 0  
Soy la vuelta 1  
Soy la vuelta 2  
Soy la vuelta 3  
Soy la vuelta 4
```

Aquí tienes el resultado de las primeras vueltas (así hasta la 100).

Como ya te he comentado anteriormente, lo utilizaremos cuando sepamos el número exacto de iteraciones: lee 15 números o calcula la media de las notas de 30 alumnos.

Otros ejemplos (te animo a probarlos y ver su resultado):

ejemplos_bucles.py ×

```
1 print("Comienzo")
2 for i in [0, 1, 2]:
3     print("Hola ", end="")
4 print()
5 print("Final")
```

ejemplos_bucles.py ×

```
1 print("Comienzo")
2 for i in [3, 4, 5]:
3     print(f"Hola. Ahora i vale {i} y su cuadrado {i ** 2}")
4 print("Final")
```

ejemplos_bucles.py ×

```
1 print("Comienzo")
2 for i in ["Alba", "Benito", 27]:
3     print(f"Hola. Ahora i vale {i}")
4 print("Final")
```

Testigos, contadores y acumuladores

En muchos programas se necesitan variables que indiquen si simplemente ha ocurrido algo (testigos), o que cuenten cuántas veces ha ocurrido algo (contadores), o que acumulen valores (acumuladores). Las situaciones pueden ser muy diversas, por lo que en este apartado simplemente se ofrecen unos ejemplos para mostrar la idea.

Testigo

Se entiende por testigo una variable que indica simplemente si una condición se ha cumplido o no. Es un caso particular de contador, pero se suele hacer con

variables lógicas en vez de numéricas (en este caso, la variable que hace de testigo es la variable *sacaste_cinco*):

```
ejemplos_bucles.py ×  
1 import random  
2  
3 print("Comienzo")  
4 sacaste_cinco = False  
5 for i in range(3):  
6     dado = random.randrange(1, 7)  
7     print(f"Tirada {i + 1}: {dado}")  
8     if dado == 5:  
9         sacaste_cinco = True  
10 if sacaste_cinco:  
11     print("Ha salido al menos un 5.")  
12 else:  
13     print("No ha salido ningún 5.")  
14 print("Final")
```

Quizá no entiendas algunos conceptos del programa:

- `import`: nos permite utilizar la librería `random`, que se utiliza para generar números aleatorios.
- `random.randrange`: genera un número aleatorio entre el 1 y el 7

Detalles importantes:

- Antes del bucle se debe dar un valor inicial al testigo (en este caso, `False`)
- En cada iteración, el programa comprueba si `dado` es 5.
- El testigo se modifica la primera vez que la variable `i` es 5.
- El testigo no cambia a `False` una vez ha cambiado `True`.
- Se podría haber utilizado un `range(1, 4)` para poder escribir la tirada escribiendo `{i}` en vez de `{i + 1}`.

Contador

Se entiende por contador una variable que lleva la cuenta del número de veces que se ha cumplido una condición. El ejemplo siguiente es un ejemplo de programa con contador (en este caso, la variable que hace de contador es la variable *cuenta_cincos*):

```
ejemplos_bucles.py ×
1 import random
2
3 print("Comienzo")
4 cuenta_cincos = 0
5 for i in range(3):
6     dado = random.randrange(1, 7)
7     print(f"Tirada {i + 1}: {dado}")
8     if dado == 5:
9         cuenta_cincos += 1
10 print(f"En total ha(n) salido {cuenta_cincos} cinco(s).")
11 print("Final")
```

Detalles importantes:

- Antes del bucle se debe dar un valor inicial al contador (en este caso, 0)
- En cada iteración, el programa comprueba si dado es 5.
- El contador se modifica sólo si la variable dado es cinco.
- El contador va aumentando de uno en uno.
- Si la frase indicando el número de cincos obtenidos se quisiera adaptar al número de cincos obtenidos (por ejemplo, "no se han obtenido cincos", "sólo se ha obtenido un cinco" o "se han obtenido XX cincos") se debería utilizar una estructura if ... elif ... else ...

Acumulador

Se entiende por acumulador una variable que acumula el resultado de una operación. El ejemplo siguiente es un ejemplo de programa con acumulador (en este caso, la variable que hace de acumulador es la variable *total*):

ejemplos_bucles.py * x

```
1 import random
2
3 print("Comienzo")
4 total = 0
5 for i in range(3):
6     dado = random.randrange(1, 7)
7     print(f"Tirada {i + 1}: {dado}")
8     total += dado
9 print(f"En total ha obtenido {total} punto(s).")
10 print("Final")
```

Detalles importantes:

- El acumulador se modifica en cada iteración del bucle (en este caso, el valor de dado se añade al acumulador total).
- Antes del bucle se debe dar un valor inicial al acumulador (en este caso, 0)
- Si la frase indicando el número de puntos obtenidos se quisiera adaptar al número de puntos obtenidos (por ejemplo, "sólo se ha obtenido un puntos" o "se han obtenido XX puntos") se debería utilizar una estructura if ... else ...

Bucles FOR anidados

Antes de continuar con el siguiente tipo de bucles, es necesario echarle un vistazo a los bucles que se encuentran dentro de otros bucles.

Uno será el externo y otro el interno.

Bucles anidados (variables independientes)

Se dice que las variables de los bucles son **independientes** cuando los valores que toma la variable de control del bucle interno **no** dependen del valor de la variable de control del bucle externo. Por ejemplo (prueba a ver lo que hace):

ejemplos_bucles.py ×

```
1 for i in [0, 1, 2]:
2     for j in [0, 1]:
3         print(f"i vale {i} y j vale {j}")
```

Bucles anidados (variables dependientes)

Se dice que las variables de los bucles son **dependientes** cuando los valores que toma la variable de control del bucle interno dependen del valor de la variable de control del bucle externo. Por ejemplo:

ejemplos_bucles.py ×

```
1 for i in [1, 2, 3]:
2     for j in range(i):
3         print(f"i vale {i} y j vale {j}")
```

En el ejemplo anterior, el bucle externo (el controlado por *i*) se ejecuta 3 veces y el bucle interno (el controlado por *j*) se ejecuta 1, 2 y 3 veces. Por ello la instrucción `print()` se ejecuta en total 6 veces.

La variable *i* toma los valores de 1 a 3 y la variable *j* toma los valores de 0 a *i*, por lo que cada vez el bucle interno se ejecuta un número diferente de veces:

- Cuando *i* vale 1, `range(i)` devuelve la lista `[0]` y por tanto el bucle interno se ejecuta una sola vez y el programa escribe una sola línea en la que *i* vale 1 (y *j* vale 0).
- Cuando *i* vale 2, `range(i)` devuelve la lista `[0, 1]` y por tanto el bucle interno se ejecuta dos veces y el programa escribe dos líneas en la que *i* vale 2 (y *j* vale 0 o 1 en cada una de ellas).
- Cuando *i* vale 3, `range(i)` devuelve la lista `[0, 1, 2]` y por tanto el bucle interno se ejecuta tres veces y el programa escribe tres líneas en la que *i* vale 3 (y *j* vale 0, 1 o 2 en cada una de ellas).

Bucle WHILE

Utilizaremos este tipo de bucles cuando no sepamos el número exacto de iteraciones que hay que realizar, siempre que se cumpla una condición: no hemos llegado al final de un fichero, no se ha escrito cierto carácter o cualquier otra condición.

Vayamos con un primer ejemplo sencillo:

<pre>ejemplos_bucles.py × 1 i = 1 2 while i <= 3: 3 print(i) 4 i += 1 5 print("Programa terminado")</pre>	<pre>Consola × >>> %Run ejemplos_bucles.py 1 2 3 Programa terminado >>></pre>
--	---

A la izquierda tienes el código y a la derecha el resultado.

La ventaja de un bucle while es que la variable de control se puede modificar con mayor flexibilidad, como en el ejemplo siguiente:

<pre>ejemplos_bucles.py × 1 i = 1 2 while i <= 50: 3 print(i) 4 i = 3 * i + 1 5 print("Programa terminado")</pre>	<pre>>>> %Run ejemplos_bucles.py 1 4 13 40 Programa terminado</pre>
--	--

Bucles infinitos

Una de las cosas que debemos evitar es la creación de bucles infinitos, que son los que nunca terminan de ejecutarse. Deben evitarse pues significan perder el control del programa.

Para interrumpir un bucle infinito, hay que pulsar la combinación de teclas `Ctrl+C`. Al interrumpir un programa se mostrará un mensaje de error similar a éste:

```
Traceback (most recent call last):
  File "ejemplo.py", line 3, in <module>
    print(i)
KeyboardInterrupt
```

Ejemplos de bucles infinitos:

```
i = 1
while i <= 10:
    print(i, end=" ")
```

```
i = 1
while i > 0:
    print(i, end=" ")
    i += 1
```

2.3 Funciones

Podemos definir una función como un código al que podemos llamar en cualquier momento para ejecutarlo. Por ejemplo, se puede crear una función que nos calcule el mayor de 3 números o que nos imprima cierta información por pantalla.

Debemos distinguir entre dos tipos de funciones. El primer tipo serían las que no necesitan argumentos. Podemos pensar que una función es una receta, en la que podemos utilizar ingredientes o no. En nuestro caso, los ingredientes se llaman parámetros. Como hemos comentado antes, una función puede encontrar el mayor de 3 números. En esta ocasión, los 3 números serían los parámetros. O una función muestra un mensaje por pantalla, en cuyo caso no harían falta parámetros.

2.3.1 Funciones sin parámetros

Antes de poder utilizarla, habrá que definirla y crear su código correspondiente.

```
ejemplos_funciones.py ×
1 def mi_informacion():
2     print('Estás en un curso de Python')
3     print('En este tema estás estudiando las funciones')
4
5 print('Vamos a probar las funciones')
6 mi_informacion()
7 print('Fin')
```

Como puedes ver, primero definimos la función (en este caso se llama `mi_informacion`) y después va su código (líneas 2 y 3). En la línea 5 comienza el programa principal.

Hay que definir la función antes de poder utilizarla y puede tener la palabra reservada `return` al final, pero no es obligatorio su uso.

Pero puede haber un problema con las variables, que se tratan en el punto siguiente

Variables en funciones

Quizá el texto que viene a continuación sea un poco lioso, pero es necesario comprenderlo para no cometer errores en el uso de las variables dentro de las funciones. En otros lenguajes se pueden definir variables dentro de las funciones, pero esto no ocurre en Python, por lo que habrá que tener cuidado.

Conflictos de nombres de variables

Como se ha comentado antes, una de las principales ventajas de las subrutinas es que permiten reutilizar el código. Pero copiar y pegar subrutinas de un programa a otro puede producir lo que se llama un **conflicto de nombres de variables**. En efecto, si la subrutina que pegamos en un programa utiliza alguna variable auxiliar para algún cálculo intermedio y resulta que el programa ya utilizaba una variable con el mismo nombre que esa variable auxiliar, los cambios en la variable que se hagan en la subrutina podrían afectar al resto del programa de forma imprevista.

Para resolver el problema de los conflictos de nombres, los lenguajes de programación limitan lo que se llama el alcance o el ámbito de las variables. Es decir, que los lenguajes de programación permiten que una variable exista únicamente en el interior de una subrutina y no afecte a otras variables de mismo nombre situadas fuera de esa subrutina. Como las subrutinas pueden contener a su vez subrutinas, se suele hablar de niveles: el nivel más alto sería el programa

principal, el siguiente nivel serían las subrutinas incluidas en el programa principal y cada vez que hay una subrutina incluida dentro de otra estaríamos bajando un nivel.

El problema es más complicado de lo que parece a primera vista, porque a menudo también nos interesará que una subrutina pueda modificar variables que estén definidas en otros puntos del programa. Así que los lenguajes de programación tienen que establecer mecanismos para aislar las variables y evitar los conflictos de nombres, pero al mismo tiempo deben permitir el acceso a las variables en los casos que así lo quiera el programador.

Aunque cada lenguaje tiene sus particularidades, el mecanismo más habitual se basa en los siguientes principios:

- cada variable pertenece a un ámbito determinado: al programa principal o a una subrutina.
- las variables son completamente inaccesibles en los ámbitos superiores al ámbito al que pertenecen
- las variables pueden ser accesibles o no en ámbitos inferiores al ámbito al que pertenecen (el lenguaje puede permitir al programador elegir o no esa accesibilidad)
- en cada subrutina, las variables que se utilizan pueden ser entonces:
 - variables locales: las que pertenecen al ámbito de la subrutina (y que pueden ser accesibles a niveles inferiores)
 - variables libres: las que pertenecen a ámbitos superiores pero son accesibles en la subrutina

Conflictos de nombres de variables en Python

Python sigue estos principios generales, pero con algunas particularidades:

- En los lenguajes tipificados, como se debe declarar las variables que se utilizan, la declaración se aprovecha para indicar si la variable pertenece a la subrutina o procede de un ámbito superior. Pero como Python no es un

lenguaje tipificado, el ámbito de pertenencia de la variable debe deducirse del programa siguiendo unas reglas que se detallan más adelante (aunque Python también permite declarar explícitamente el ámbito en los casos en que se quiera un ámbito distinto al determinado por las reglas).

- Python distingue tres tipos de variables: las variables locales y dos tipos de variables libres (globales y no locales):
 - variables locales: las que pertenecen al ámbito de la subrutina (y que pueden ser accesibles a niveles inferiores)
 - variables globales: las que pertenecen al ámbito del programa principal.
 - variables no locales: las que pertenecen a un ámbito superior al de la subrutina, pero que no son globales.

Si el programa contiene solamente funciones que no contienen a su vez funciones, todas las variables libres son variables globales. Pero si el programa contiene una función que a su vez contiene una función, las variables libres de esas "subfunciones" pueden ser globales (si pertenecen al programa principal) o no locales (si pertenecen a la función).

- Para identificar explícitamente las variables globales y no locales se utilizan las palabras reservadas `global` y `nonlocal`. Las variables locales no necesitan identificación. La palabra reservada `nonlocal` se introdujo en Python 3 (PEP 3104).

A continuación, se detallan las reglas y situaciones posibles, acompañadas de ejemplos.

Variables locales

- Si no se han declarado como globales o no locales, las variables **a las que se asigna valor** en una función se consideran variables **locales**, es decir, sólo existen en la propia función, incluso cuando en el programa exista una variable con el mismo nombre, como muestra el siguiente ejemplo:

ejemplos_funciones.py ×

```
1 def subrutina():
2     a = 2
3     print(a)
4     return
5
6 a = 5
7 subrutina()
8 print(a)
```

Consola ×

```
>>> %Run ejemplos_funciones.py
2
5
>>>
```

A la izquierda tienes el código que probar y a la derecha el resultado

- Las variables locales sólo existen en la propia función y no son accesibles desde niveles superiores, como puede verse en el siguiente ejemplo:

ejemplos_funciones.py ×

```
1 def subrutina():
2     a = 2
3     print(a)
4     return
5
6 subrutina()
7 print(a)
```

Consola ×

```
2
Traceback (most recent call last):
  File "C:\Users\rosmo\ejemplos_funciones.py", line 7, in <module>
    print(a)
NameError: name 'a' is not defined
```

Como puedes ver, la variable `a` da error ya que ha sido definida en la función y no se puede utilizar fuera de ella.

Variables libres globales o no locales

Si a una variable **no se le asigna valor** en una función, Python la considera **libre** y busca su valor en los niveles superiores de esa función, empezando por el

inmediatamente superior y continuando hasta el programa principal. Si a la variable se le asigna valor en algún nivel intermedio la variable se considera **no local** y si se le asigna en el programa principal la variable se considera **global**, como muestran los siguientes ejemplos:

ejemplos_funciones.py ×

```

1 def subrutina():
2     print(a)
3     return
4
5 a = 5
6 subrutina()
7 print(a)

```

Consola ×

```

>>> %Run ejemplos_funciones.py
5
5
>>>

```

A continuación, veremos las variables globales, que se definen en una función y se pueden utilizar en el resto del programa.

Variables declaradas global o nonlocal

Si queremos asignar valor a una variable en una subrutina, pero no queremos que Python la considere local, debemos declararla en la función como global o nonlocal, como muestran los ejemplos siguientes:

Ejemplo de variable declarada global:

ejemplos_funciones.py ×

```

1 def subrutina():
2     global a
3     print(a)
4     a = 1
5     return
6
7 a = 5
8 subrutina()
9 print(a)

```

Consola ×

```

>>> %Run ejemplos_funciones.py
5
1
>>>

```

Ejemplo de variable declarada nonlocal:

```
ejemplos_funciones.py * ×
1 def subrutina():
2     def sub_subrutina():
3         nonlocal a
4         print(a)
5         a = 1
6         return
7
8     a = 3
9     sub_subrutina()
10    print(a)
11    return
12
13    a = 4
14    subrutina()
15    print(a)
```

```
Consola ×
>>> %Run ejemplos_funciones.py
3
1
4
```

El programa y su resultado.

Ejemplo de variable declarada nonlocal no definida

```
ejemplos_funciones.py ×
1 def subrutina():
2     def sub_subrutina():
3         nonlocal a
4         print(a)
5         a = 1
6         return
7
8     sub_subrutina()
9     print(a)
10    return
11
12    a = 4
13    subrutina()
14    print(a)
```

```
Consola ×
>>> %Run ejemplos_funciones.py
Traceback (most recent call last):
  File "C:\Users\rosmo\ejemplos_funciones.py", line 3
    nonlocal a
    ^^^^^^^^^
SyntaxError: no binding for nonlocal 'a' found
```

2.3.2 Funciones con parámetros

El problema que tienen las funciones sin parámetros es que difícilmente son reutilizables en otros programas.

Veamos un ejemplo de función con parámetros:

```
ejemplos_funciones_con_parametros.py ×  
1 def escribe_media(x, y):  
2     media = (x + y) / 2  
3     print(f"La media de {x} y {y} es: {media}")  
4     return  
5  
6 a = 3  
7 b = 5  
8 escribe_media(a, b)  
9 print("Programa terminado")
```

Esta función puede ser utilizada con más flexibilidad que la del ejemplo anterior, puesto que el programador puede elegir a qué valores aplicar la función.

Pero esta función tiene todavía un inconveniente. Como las variables locales de una función son inaccesibles desde los niveles superiores, el programa principal no puede utilizar la variable "media" calculada por la función y tiene que ser la función la que escriba el valor. Pero eso complica la reutilización de la función, porque, aunque es probable que en otro programa nos interese una función que calcule la media, es más difícil que nos interese que escriba el valor nada más calcularlo.

Para evitar ese problema, las funciones pueden devolver valores, es decir, pueden enviar valores al programa o función de nivel superior. De esa manera, las funciones se pueden reutilizar más fácilmente, como muestra el ejemplo siguiente:

```
ejemplos_funciones_con_parametros.py ×
1 def calcula_media(x, y):
2     resultado = (x + y) / 2
3     return resultado
4
5 a = 3
6 b = 5
7 media = calcula_media(a, b)
8 print(f"La media de {a} y {b} es: {media}")
9 print("Programa terminado")
```

El único inconveniente que tiene esta última función es que únicamente calcula la media de 3 y 5, por lo que calculara el valor medio de cualquier cantidad de valores. Lo vemos en el siguiente ejemplo:

```
ejemplos_funciones_con_parametros.py ×
1 def calcula_media(*args):
2     total = 0
3     for i in args:
4         total += i
5     resultado = total / len(args)
6     return resultado
7
8 a, b, c = 3, 5, 10
9 media = calcula_media(a, b, c)
10 print(f"La media de {a}, {b} y {c} es: {media}")
11 print("Programa terminado")
```

Para que una función tenga más flexibilidad, sería recomendable que pudiesen devolver varios valores simultáneamente. Eso lo conseguimos de la siguiente manera (si te fijas, deben coincidir los valores que hay después del return, media y desviación, con los que hay cuando se llama la función):

ejemplos_funciones_con_parametros.py ×

```

1 def calcula_media_desviacion(*args):
2     total = 0
3     for i in args:
4         total += i
5     media = total / len(args)
6     total = 0
7     for i in args:
8         total += (i - media) ** 2
9     desviacion = (total / len(args)) ** 0.5
10    return media, desviacion
11
12 a, b, c, d = 3, 5, 10, 12
13 media, desviacion_tipica = calcula_media_desviacion(a, b, c, d)
14 print(f"Datos: {a} {b} {c} {d}")
15 print(f"Media: {media}")
16 print(f"Desviación típica: {desviacion_tipica}")
17 print("Programa terminado")

```

Y éste es el resultado después de la ejecución:

Consola ×

```

>>> %Run ejemplos_funciones.py
Datos: 3 5 10 12
Media: 7.5
Desviación típica: 3.640054944640259
Programa terminado

```

Conflictos entre nombres de parámetros y nombre de variables globales

En Python no se producen conflictos entre los nombres de los parámetros y los nombres de las variables globales. Es decir, el nombre de un parámetro puede coincidir o no con el de una variable global, pero Python no los confunde: en el ámbito de la función el parámetro hace siempre referencia al dato recibido y no a la variable global y los programas producen el mismo resultado.

Eso nos facilita reutilizar funciones en otros programas sin tener que preocuparnos por este detalle.

Aunque, como siempre, dependiendo de si a la función se le envía como parámetro un objeto mutable o inmutable, la función podrá modificar o no al objeto. En los dos siguientes ejemplos, el parámetro de la función ("b") se llama igual que una de las dos variables del programa principal. En los dos ejemplos se llama dos veces a la función, enviando cada vez una de las dos variables ("a" y "b").

- **Ejemplo de conflicto entre nombre de parámetro y nombre de variable global. Objeto mutable**

Como en este caso las variables son listas (objetos mutables, que estudiaremos en el siguiente apartado del tema), la función modifica la lista que se envía como argumento: primero se modifica la lista "a" y a continuación la lista "b". La lista modificada no depende del nombre del parámetro en la función (que es "b"), sino de la variable enviada como argumento ("a" o "b").

```
ejemplos_funciones.py ×
1 def cambia(b):
2     b += [5]
3     return
4
5 a, b = [3], [4]
6 print(f"Al principio          : a = {a} b = {b}")
7 cambia(a)
8 print(f"Después de cambia(a): a = {a} b = {b}")
9 cambia(b)
10 print(f"Después de cambia(b): a = {a} b = {b}")
11 print("Programa terminado")
```

Y se obtiene como resultado al ejecutar el programa:

```
Consola ×
>>> %Run ejemplos_funciones.py

Al principio          : a = [3] b = [4]
Después de cambia(a): a = [3, 5] b = [4]
Después de cambia(b): a = [3, 5] b = [4, 5]
Programa terminado
```

- **Ejemplo de conflicto entre nombre de parámetro y nombre de variable global. Objeto inmutable**

Como en este caso las variables son números enteros (objetos inmutables), la función no puede modificar los números que se envían como argumentos, ni la variable "a" ni la variable "b".

```
ejemplos_funciones.py x
1 def cambia(b):
2     b += 1
3     return
4
5 a, b = 3, 4
6 print(f"Al principio          : a = {a} b = {b}")
7 cambia(a)
8 print(f"Después de cambia(a): a = {a} b = {b}")
9 cambia(b)
10 print(f"Después de cambia(b): a = {a} b = {b}")
11 print("Programa terminado")
```

Pulsamos F5 y obtenemos el resultado:

```
Consola x
>>> %Run ejemplos_funciones.py
Al principio          : a = 3 b = 4
Después de cambia(a): a = 3 b = 4
Después de cambia(b): a = 3 b = 4
Programa terminado
```

Paso por valor o paso por referencia

En los lenguajes en los que las variables son "cajas" en las que se guardan valores, cuando se envía una variable como argumento en una llamada a una función suelen existir dos posibilidades:

- **paso por valor:** se envía simplemente el valor de la variable, en cuyo caso la función no puede modificar la variable, pues la función sólo conoce su valor, pero no la variable que lo almacenaba.

- paso por referencia: se envía la dirección de memoria de la variable, en cuyo caso la función sí que puede modificar la variable.

En Python no se hace ni una cosa ni otra. En Python cuando se envía una variable como argumento en una llamada a una función lo que se envía es la referencia al objeto al que hace referencia la variable. Dependiendo de si el objeto es mutable o inmutable, la función podrá modificar o no el objeto.

- En el ejemplo siguiente, la variable enviada a la función es una variable que hace referencia a un objeto inmutable.

ejemplos_funciones.py ×

```

1 def aumenta(x):
2     print(id(x))
3     x += 1
4     print(id(x))
5     return x
6
7 a = 3
8 print(id(3), id(4))
9 print(id(a))
10 print(aumenta(a))
11 print(a)
12 print(id(a))

```

Consola ×

```

2175061459248
2175061459248
2175061459280
4
3
2175061459248

```

En el ejemplo siguiente, la variable enviada a la función es una variable que hace referencia a un objeto mutable.

ejemplos_funciones.py ×

```

1 def aumenta(x):
2     print(id(x))
3     x += [1]
4     print(id(x))
5     return x
6
7 a = [3]
8 print(id(a))
9 print(aumenta(a))
10 print(a)
11 print(id(a))

```

Consola ×

```

1580562679104
1580562679104
1580562679104
[3, 1]
[3, 1]
1580562679104

```

2.4 Ejercicios

A continuación, te planteo una serie de ejercicios para repasar los conceptos aprendidos hasta el momento.

1.- Escribe un programa que pida primero un número par y luego un número impar (positivos o negativos). En caso de que uno o los dos valores no sea correcto, se mostrará un único aviso.

2.- Escribe un programa que pida primero un número par (positivo o negativo) y si el valor no es correcto, muestre un aviso. Si el valor es correcto, pedirá un número impar (positivo o negativo) y si el valor no es correcto, mostrará un aviso.

3.- Escribe un programa que pida primero un número par y a continuación un número impar (positivos o negativos). En caso de que uno o los dos valores no sean correctos, se mostrarán uno o dos avisos.

4.- Escribe un programa que pida primero un número par y a continuación un número impar (positivos o negativos). En cada petición, si el valor no es correcto se mostrará un aviso.

5.- Escribe un programa que pregunte una y otra vez si desea continuar con el programa, siempre que se conteste exactamente sí (en minúsculas y con tilde).

6.- Escriba un programa que pregunte una y otra vez si desea terminar el programa, siempre que se conteste exactamente N (en mayúsculas).

2.5 Métodos

Los métodos se estudian en el siguiente apartado, el de la programación a objetos y las clases. Aquí se hará una breve introducción, ya que son muy parecidos a las funciones.

Las clases es una forma de empaquetar (unir) datos y funciones juntos. Como ejemplo, una cadena de caracteres es una clase, que tienen ejemplos concretos (objetos) a los que se pueden aplicar unos métodos (obtener su longitud) y además tiene unas propiedades.

Entonces, no vamos a profundizar mucho en los métodos ya que los analizaremos en la unidad 3.

2.6 Listas, tuplas y diccionarios.

Este tipo de datos los utilizaremos cuando sea necesario guardar gran cantidad de información.

2.6.1 Listas

Las listas son una colección de datos que resultan de gran importancia y nos ahorran mucho trabajo a la hora de realizar un programa. Pueden contener datos de diferentes tipos. En una misma lista puede haber texto, números, valores lógicos, etc. Nosotros podremos referirnos a estos datos mediante un índice. Para acceder al elemento utilizaremos corchetes.

```
listas.py ×  
1 lista = ['Albacete', 1940, 'Goles', 66, True]  
2 print('Primer elemento ',lista[0])
```

Resultado:

```
Consola ×  
>>> %Run listas.py  
Primer elemento Albacete
```

Recuerda que el primer elemento es el 0

Las listas son uno de los elementos más utilizados en Python y es uno de los puntos en los que destaca.

A continuación, tienes un ejemplo que puedes probar, con diferentes métodos que tienen las listas.

```
listas.py * x
1 def recorre_lista(l):
2     for x in l:
3         print(x)
4
5 lista = ['Albacete', 1940, 'Goles', 66, True]
6
7 recorre_lista(lista)
8
9 lista.append('Adios')
10 print('Elemento añadido')
11 recorre_lista(lista)
12 #Invertimos la lista
13 lista.reverse()
14 recorre_lista(lista)
15 #Añadimos el elemento Carlos Belmonte en la posición 1
16 lista.insert(1, 'Carlos Belmonte')
17 recorre_lista(lista)
18 #Mostramos los elementos del 1 al 4
19 recorre_lista(lista[1:4])
```

Un aspecto importante es que podemos acceder a los caracteres de una cadena de texto como accedemos a las listas.

Para complicarlo un poco más, una lista puede contener listas (que a su vez pueden contener listas, que a su vez etc.):

```
>>> persona1 = ["Ana", 25]
>>> persona2 = ["Benito", 23]
>>> lista = [persona1, persona2]
>>> lista
[['Ana', 25], ['Benito', 23]]
```

Se pueden concatenar dos listas utilizando la operación suma:

```
>>> lista1 = [10, 20, 30, 40]
>>> lista2 = [30, 20]
>>> lista = lista1 + lista2 + lista1
>>> lista
[10, 20, 30, 40, 30, 20, 10, 20, 30, 40]
```

Tampoco se pueden hacer referencia a elementos fuera de la lista, es decir, si hay 4 elementos no podremos acceder a la posición 4 (recuerda que sería de la 0 a la 3)

La palabra reservada del

Para borrar elementos de la lista podemos utilizar la palabra reservada del:

```
letras = ["A", "B", "C", "D", "E", "F", "G", "H"]
>>> del letras [4] # Elimina la sublista ['E']
>>> letras
['A', 'B', 'C', 'D', 'F', 'G', 'H']
```

Te dejo que investigues por tu cuenta el resto de métodos de las listas ya que incluso hay libros enteros para trabajar solo con las listas.

2.6.1.1 Ejercicios de listas

1. Escribir un programa que almacene las asignaturas de un curso (por ejemplo Matemáticas, Física, Química, Historia y Lengua) en una lista y la muestre por pantalla.

2. Escribir un programa que pregunte al usuario los números ganadores de la lotería primitiva, los almacene en una lista y los muestre por pantalla ordenados de menor a mayor.

3. Escribir un programa que permita crear una lista de palabras. Para ello, el programa tiene que pedir un número y luego solicitar ese número de palabras para crear la lista. Por último, el programa tiene que escribir la lista.

4. Escribir un programa que permita crear una lista de palabras y que, a continuación, pida una palabra y diga cuántas veces aparece esa palabra en la lista.

2.6.2 Tuplas

Una **tupla** es un conjunto ordenado e inmutable de elementos del mismo o diferente tipo. Son muy parecidas a las listas, pero se diferencian en que las tuplas son inmutables.

```
mi_tupla=(1,2,3,4)
```

```
print(mi_tupla)
```

Normalmente se declaran con paréntesis y separados por comas, aunque podríamos declararlos sin paréntesis.

```
mi_tupla=1,2,3,4
```

Lo de ser inmutables significa que una vez que asignemos su valor, éste no puede ser modificado. Si intentamos modificar algún valor nos devolverá un error.

Un parecido con las listas es que las tuplas también pueden ser anidadas.

```
tuplas.py ×  
1 mi_tupla = 1, 2, ('a', 'b'), 3  
2 print(mi_tupla)           #(1, 2, ('a', 'b'), 3)  
3 print(mi_tupla[2][0])    #a
```

Además, podemos convertir una lista en una tupla:

```
tuplas.py ×  
1 mi_lista = [1, 2, 3]  
2 mi_tupla = tuple(mi_lista)  
3 print(type(mi_tupla)) #<class 'tuple'>  
4 print(mi_tupla)      #(1, 2, 3)
```

De la siguiente forma podemos recorrer una tupla:

```
tuplas.py ×  
1 mi_tupla = [1, 2, 3]  
2 for elemento in mi_tupla:  
3     print(elemento) #1, 2, 3
```

Entre los métodos de las tuplas podemos destacar:

```
tuplas.py ×  
1 mi_tupla = (1, 1, 1, 3, 5)  
2 print(mi_tupla.count(1)) #3
```

Count cuenta las veces que aparece el elemento que le pasamos como parámetro en una tupla

El método index() también acepta un segundo parámetro opcional, que indica a partir de que índice empezar a buscar el objeto.

```
tuplas.py * ×  
1 mi_tupla = (7, 7, 7, 3, 5)  
2  
3 print(mi_tupla.index(7,2))
```

Devuelve un 2

Por último, vamos a estudiar los diccionarios.

2.6.3 Diccionarios

Son una estructura de datos en las que almacenamos información, con el formato de llave y valor. Aquí no hay un orden, primer elemento, segundo elemento.. NO

hay un índice numérico. Hay parejas de PALABRAS CLAVE y EL VALOR QUE TOMAN. (keys and values). Pero veámoslo mejor con un ejemplo:

```
diccionarios.py ×
1 usuario1 = {
2     "Nombre": "Pepe",
3     "Edad": 75,
4     "Documento": 123456789
5 }
6 print(usuario1)
```

```
Consola ×
>>> %Run diccionarios.py
{'Nombre': 'Pepe', 'Edad': 75, 'Documento': 123456789}
```

Si queremos conocer el nombre del usuario, escribiremos:

```
print(usuario1['Nombre'])
```

Veamos otro ejemplo:

```
cumples = {'Juan': '2 Marzo', 'María': '5 Junio', 'Luisa': '15 Septiembre'}
```

Para saber cuándo cumple los años Luisa,:

```
print(cumples['Luisa'])
```

Si tenemos el siguiente diccionario:

```
mi_coche = {'marca': 'Renault', 'modelo': '7', 'year': 1975}
```

Para modificar el modelo, podríamos escribir:

```
mi_coche['modelo']='8'
```

Cuando queremos recorrer el diccionario, podríamos cometer el siguiente error:

```
for x in mi_coche:
    print(x)
```

marca
modelo
year

Para ver realmente los valores deberíamos escribir:

```
for x in mi_coche:
```

```
    print(mi_coche[x])
```

Renault
8
1975

Recuerda que los diccionarios también son OBJETOS, así que tienes a tu disposición un conjunto de funciones (métodos) que ya están programadas y sólo tienes que “invocar” para obtener lo que desees. Algunos ejemplos:

- clear() elimina todos los elementos del diccionario
- copy() Devuelve una copia del diccionario
- fromkeys() Devuelve un diccionario con las claves y valores especificados
- get() devuelve el valor de la clave especificada
- items() Devuelve una lista que contiene la tupla A para cada par de valores clave
- keys() Devuelve una lista que contiene las claves del diccionario
- pop() elimina el elemento con la clave especificada
- popitem() elimina el último par de valores de llave insertado
- setdefault() Devuelve el valor de la clave especificada. Si la clave no existe: inserta la clave, con el valor especificado
- update() actualiza el diccionario con los pares de valor clave especificados
- values() Devuelve una lista de todos los valores en el diccionario

Ejemplos:

```
mi_coche.pop('marca')
```

Muestra la marca por pantalla y la quita del diccionario.

A continuación tienes un ejemplo de diccionario de diccionarios, ya que al igual que en las listas o tuplas, se pueden anidar:

```
myfamily = {
    "child1" : {
        "name" : "Emil",
        "year" : 2004
    },
    "child2" : {
        "name" : "Tobias",
        "year" : 2007
    }
}
```

```
},  
"child3" : {  
  "name" : "Linus",  
  "year" : 2011  
}  
}
```

child1 es una clave del diccionario “grande” y su valor es el diccionario “pequeño”
con las claves/valores name:Emil, year:2004

child2 es otra clave y child3 es otra clave, mientras que los diccionarios

```
{"name": "Tobias", "year": 2007} y {"name": "Linus", "year": 2011}
```

Son sus valores respectivos.

3 Resumen

Como ya te comenté, si entiendes este tema perfectamente, ya tendrás grandes conocimientos sobre Python. Qué hemos aprendido:

- Los tipos de datos que existen en Python
- Las estructuras de control de flujo.
- Qué son las funciones (con y sin parámetros)
- Diferencias entre funciones y métodos
- Cuidado con las variables y su ámbito
- Estructuras de datos:
 - Listas
 - Tuplas
 - Diccionarios

Nos vemos en el siguiente tema, donde aprenderemos todo sobre la Programación Orientada a Objetos (POO).



¡Enhorabuena!

Has finalizado el estudio de esta unidad



comunicación@bilib.es